

Dual-core machine learning accelerator for attention mechanism

Shreeya Ajay Bhonsle (A69040642) , Chi-Han Chiu (A69036037) , Nikhita Neelakanta (A69041742) ,
Ming-Yang Wu (A16504833) , Bing-Cheng Chiang (A69040193) , Wen-Chieh Lo (A69042225)

Abstract—This report presents the design and physical implementation of a dual-core machine learning accelerator tailored for the attention mechanisms in modern transformer models. The architecture features an asynchronous clocking scheme that distributes matrix multiplication and normalization across a dual-core setup. A hardware-optimized, sparsity-aware Softmax (SFP) engine is introduced, which utilizes cross-core sum exchange and bypasses redundant computations to improve energy efficiency. To overcome critical timing bottlenecks, we applied rigorous architectural optimizations including a pipelined long divider and adder tree reducing the critical path delay from over 24 ns to under 1.0 ns and shrinking the full-chip standard cell area by 47.8%. Furthermore, we developed a highly scalable, Makefile-driven verification framework anchored by a custom .mingu shell. This framework seamlessly bridges behavioral RTL simulations with large-scale, automated post-PnR gate-level simulations. Validated through a TSMC 65nm hierarchical physical design flow, the accelerator achieves zero timing violations and demonstrates fully functional, tape-out-ready physical layouts.

I. INTRODUCTION

Attention mechanisms form the computational backbone of modern transformer models, but their reliance on intensive matrix multiplications (MatMul) and row-wise Softmax normalizations presents significant throughput and memory bottlenecks. To address these challenges, this project proposes a dual-core hardware accelerator designed specifically for efficient attention computation.

The architecture partitions the workload across two independent cores communicating via asynchronous FIFOs, enabling flexible cross-domain scheduling. To alleviate the Softmax bottleneck, we developed a Sparsity-Aware Softmax (SFP) engine that dynamically skips computations for insignificant attention scores, reducing dynamic power and latency. Given the complexity of the asynchronous, dual-core architecture, we adopted a rigorous, incremental design methodology progressing from basic single-core RTL to a tape-out-ready, full-chip hierarchical physical layout. This report details the architectural innovations and the physical implementation milestones that achieved zero timing violations under a 1.0 ns clock constraint.

II. PROPOSED ARCHITECTURE

A. Dual-Core System & Asynchronous Clocking

To maximize throughput, the accelerator is partitioned into two distinct processing cores (Core 0 and Core 1). Each core operates in its own clock domain (`clk0` and `clk1`) to simulate realistic system-on-chip (SoC) environments where

different functional blocks might run at different frequencies. Data and control signals are securely transferred between the two clock domains using asynchronous FIFOs (Async FIFO). This cross-domain communication (CDC) ensures that matrix chunks processed by one core can be synchronized with the other core without meta-stability issues, particularly during the global sum accumulation required for Softmax normalization.

B. Core Controller & Data flow

Each core features an independent controller and a memory map (`reg_map`) to handle top-level configurations. The controller manages the data flow between the Query Memory (QMEM), Key Memory (KMEM), the MAC array, and the SFP engine based on the defined operation mode (`op_mode`):

- **Mode 1 (MatMul Only):** The core multiplies data from QMEM and KMEM using the MAC array, and directly stores the result into the Partial Sum Memory (PMEM).
- **Mode 0 (MatMul + Normalization):** Data from QMEM and KMEM are multiplied, and the resulting partial sums are fed into the SFP engine. The normalized outputs are then written back to either PMEM or KMEM, dictated by `sfp_write_to_pmem` and `sfp_write_to_kmem` flags.

The controller automatically generates status flags (e.g., `qmem_free`) to notify upper-level modules when memory banks are available for new data, enabling efficient cycle-saving data prefetching.

C. Sparsity-Aware Softmax (SFP) Engine

The SFP row block is responsible for normalizing one row of the MAC array outputs. The normalization process is divided into two pipelined phases:

- 1) **Accumulation (acc):** When `acc_start` is pulsed, the engine computes the row energy $S_i = \sum_j |sfp_in[j]|$ using an adder tree. The sum is temporarily stored in an internal FIFO, and `acc_done` is asserted upon completion.
- 2) **Division (div):** During the division phase, triggered by `div_start` (which is gated by a `div_busy` signal to prevent overlapping operations), the engine retrieves the local sum from the FIFO and adds it to the sum from the other core (`sum_in`). It then computes the normalized output:

$$sfp_div_out[c] = \text{trunc} \left(\frac{|sfp_in[c]| \ll out_shift}{S_{local} + sum_in} \right) \quad (1)$$

Upon finishing the division, a `div_done` pulse is generated to indicate that the normalized results are valid.

To further optimize performance, a sparsity gating mechanism is implemented. If the computed row energy S_i is strictly less than a configurable `SFP_THRESHOLD`, the engine bypasses the division and outputs all zeros. This effectively filters out negligible attention scores, conserving dynamic power.

III. STEP-BY-STEP IMPLEMENTATION AND VERIFICATION

To systematically conquer the design complexity, the project was executed in six sequential milestones.

A. Step 1: Single Core RTL, Synthesis & PnR

The baseline single-core design was implemented and verified first. This step established the core infrastructure, including the MAC array and the basic controller. Post-PnR layout, GLS waveform, and terminal output for the single-core design are shown below. Table III summarizes power, WNS, and DRC across implementation steps, including Step 1.

To validate the flexibility of the single-core design, we separately tested the QK product and the V-Norm product using the same computation path. This demonstrates that the design can support both stages of the attention computation, providing a verified foundation for later steps where these operations are connected into a more complete pipeline.

In addition, minor modifications were made to the professor-provided testbench to align with our implementation. Specifically, the DUT was updated from `fullchip` to `core`, along with adjustments to parameter settings and data packing format. An automatic golden result comparison (PASS/FAIL) mechanism was also added. These changes improve verification efficiency while preserving the intended functionality. Figure 13 shows the terminal output demonstrating the golden result comparison mechanism, while Figure 6 presents the corresponding waveform of the design.

B. Step 2: Output Normalisation

The single-core MAC array was integrated with the SFP (Softmax with Fixed-Point) row block. The SFP block normalizes the partial sums row by row in two phases: *accumulation* and *division*.

During the **accumulation** (`acc`) phase, the block calculates the row energy by summing the absolute values of the incoming partial sums from the MAC array ($S_i = \sum |sfp_in[j]|$).

In the **division** (`div`) phase, each absolute partial sum is divided by the global row sum to produce the normalized output. To support multi-core scaling, the global sum combines the local core's sum (S_i) with the sum from the other core (`sum_in`).

Since the theoretical output of the normalization falls within the range $[0, 1]$, integer division would yield zero for almost all values. To address this, a fixed-point scaling mechanism is introduced using the `out_shift` parameter. The numerator is logically left-shifted by `out_shift` bits before division:

$$sfp_div_out[c] = \text{truncate} \left(\frac{|sfp_in[c]| \ll out_shift}{S_i + sum_in} \right)$$

This shift defines the fractional precision of the output. For example, a default `out_shift = 7` allocates 7 bits for the fractional part, mapping the $[0, 1]$ range to $[0, 128]$ in the 8-bit output. Behavioral simulations verified that these calculations correctly normalize the partial sums. Figure 7 shows the simulation waveform demonstrating the functional correctness of the Step 2 design, with the output matching the expected normalized values.

C. Step 3: Hierarchical Synthesis of Single Core

Memory macros (SRAMs for QMEM, KMEM, and PMEM) were incorporated into the core. A hierarchical synthesis and PnR flow was adopted to manage the physical placement of the SRAM macros alongside standard cells. Each SRAM was synthesized and placed-and-routed independently with the top metal layer restricted to M4, with data input pins on the bottom edge and output pins on the top edge. The extracted Liberty timing models and LEF files from SRAM PnR were used during core-level synthesis and PnR, with SRAM instances treated as black boxes via the `dont_touch` attribute. Gate-level simulation confirmed functional correctness with zero mismatches across all 8×8 QK and VN output values.

D. Step 4: Hierarchical Synthesis of Dual-Core

Two single-core instances were integrated into a top-level fullchip module, with each core processing 8 of the 16 attention vectors in parallel. Cross-domain partial sum exchange for global normalisation was implemented via a custom Gray-code asynchronous FIFO with a sample-and-hold register at each core input. Both a lockstep (single clock) and a dual-clock (1.0 ns / 1.2 ns) configuration were validated at gate level against software-computed golden references, with the dual-clock mode included as an extension beyond the base deliverable. Table III captures the power, WNS, and DRC measurements. Figure 3 shows the dual-core fullchip layout. Figures 9 and 10 present the GLS waveforms for non-lockstep and lockstep operation, respectively. Figures 15 and 16 show the corresponding terminal outputs.

E. Step 5: Architecture Optimisations

Initial synthesis results highlighted a massive timing violation (Worst Negative Slack ≈ -23.9 ns) primarily due to deep combinational logic in the SFP row's absolute sum and division calculations. To achieve timing closure for a 1.0 ns clock period, we added 3-stage pipeline in the accumulation and also added `-retime` to meet the timing requirement:

- **Divider Optimization:** Previous synthesis timing reports indicate that the native Verilog / divider is the bottleneck of achieving 0 WNS. Therefore, we implemented 2 possible upgrades of our division module.

- 1) 6-stage pipelined long divider
- 2) 10-cycle MCP native Verilog / divider

Besides, the 2 designs share a common handshake interface with the core (`div_start`, `div_busy`, `div_done`); macros (`SFP_LONGDIV`, `SFP_MCP`,

TABLE I: Timing Optimization Synthesis Results (Target Period = 1.0 ns)

Design	Before Repipelining		After Repipelining		Area Reduction
	Area (μm^2)	WNS (ns)	Area (μm^2)	WNS (ns)	
Fullchip	557,909	-9.11	291,243	+0.04	-47.8%
MAC Array	116,808	-1.61	69,490	+0.00	-40.5%
SFP Row	53,864	-23.92	24,698	+0.00	-54.1%

VANILLA) are also defined to compile and synthesis the designs with ease.

Results. Both designs passed the Hold/Setup time check and the gate-level simulation. Nonetheless, we have to select the optimal design for PnR. Hence, we compare the synthesis result in Figure 19. The 6-staged pipelined long division hardware outperforms the 10 cycle MCP Verilog / divider with ($\sim 23\%$) lower area, slightly lower power, and much shorter synthesis time (10 min vs. 45 min). Thus, we selected the pipelined long divider for PnR.

- **Pipelining - Adder Tree Optimization:** The row energy accumulator was redesigned as a 3-stage pipelined reduction tree. After the 8 parallel multipliers, one pipeline stage captures the multiplication results, the next stage performs four pairwise additions, and the final stage reduces the partial sums to produce the accumulated row energy. Table I presents a before-and-after area and timing comparison for the repipelined blocks (fullchip, MAC array, and SFP row).
- **Chip Interface Design and Internal Control:** The chip interface is redesigned not only to remove redundant signals, but also to hide unnecessary internal details. The goal is to provide an intuitive and self-consistent interface, allowing users to operate the chip without understanding internal dataflow or timing dependencies. Each core is independently controlled, and we describe usage from the perspective of a single core.

Example Flow (MULT+NORM+READ): Write Q/K via `inst_ext` $\rightarrow$ set mode $\rightarrow$ assert start (locks memory) $\rightarrow$ compute $\rightarrow$ unlock $\rightarrow$ read PMEM. The above example involves external memory instructions and mode configuration. Refer to Figure 21 for details.

Design Insights.

- Flexible data loading; fixed execution schedule, simplifying hardware control while preserving user flexibility.
- Deterministic memory locking with hardware-enforced protection, eliminating user-side synchronization.
- Unified interface (4 commands) with mode-driven dataflow, reducing control complexity and enabling data reuse.
- Implicit pipeline support, allowing overlap of computation and data loading for higher throughput.
- **Normalization Block Protocol:** The Alpha implementation exposes a two-phase, row-local transaction. For each row, the partial-sum vector on `sfp_in` is valid for a single cycle coincident with a one-cycle `acc_start`

pulse; the same vector must be reapplied for one cycle coincident with `div_start`. The accumulation phase for a given row completes before division begins for that row (internal FIFO ordering). The cross-core partial sum `sum_in` is asserted on the cycle immediately following `div_start`, modeling a one-cycle read delay on the interconnect. A new `div_start` is permitted only when `div_busy` is low, excluding overlapping divide operations. The normalized result `sfp_div_out` is qualified by a one-cycle `div_done` pulse. Division uses $S_{\text{local}} + \text{sum_in}$ as divisor, where S_{local} is the row energy from the acc phase; the block outputs all zeros if $S_i < \text{SFP_THRESHOLD}$ or if the input row is identically zero.

- **Dual-Core Verification (Lockstep and Non-Lockstep).** Functional correctness of the dual-core integration was verified in both lockstep (single shared clock) and non-lockstep (1.0 ns / 1.2 ns asynchronous) configurations at gate level, demonstrating robustness of the asynchronous CDC protocol across both operating modes.

These architectural optimizations successfully between Step 4 and 5 reduced the WNS (Setup) from ~ 9.106 ns down to 0.001 ns, and WNS (Hold) from eliminating ~ 4.713 ns down to 0.037 ns, all timing violations. The Power increased from 29.68 mW to 54.21 mW. Table III provides the comparison parameters for all steps.

F. Step 6: Sparsity-Aware Attention

The final milestone validated the sparsity-aware feature. By injecting highly sparse matrices, we confirmed that the SFP engine correctly identifies rows below `SFP_THRESHOLD` and outputs zeros, bypassing unnecessary long division cycles.

Sparsity Definition and Threshold Calibration. We define the row-wise accumulation as:

$$S_i = \sum_j |(QK^T)_{i,j}|. \quad (2)$$

A row is skipped if $S_i < T$, where T corresponds to `SFP_THRESHOLD`. To achieve a target sparsity of 30%, T is obtained from the empirical distribution of S_i using Monte Carlo simulation over 10,000 datasets:

$$T = \text{Percentile}_{30}(S), \quad (3)$$

ensuring approximately 30% of rows satisfy $S_i < T$. The sparsity is defined as:

$$\text{sparsity} = \frac{1}{N} \sum_i \mathbf{1}(S_i < T). \quad (4)$$

RTL Skip Mechanism and Verification. In RTL, the accumulation from both cores is combined as `sum_2core`,

and the skip condition is evaluated via `sum_2core < SFP_THRESHOLD`. When triggered, the divisor is gated to zero (`sum_2core_gated`) and fed into the divider (e.g., `div_longdiv`), allowing its FSM to bypass computation and force zero outputs without modifying the control flow. The same criterion is replicated in the testbench to count skipped rows and compute sparsity. Figure 5 shows the corresponding layout.

Results. Using 10,000 sample in `threshold.py` searching threshold, the threshold for sparsity we get is 169503. With this threshold, we test test with 1000 other random generated data ,and the measured sparsity we get is 34.975%, close to the 30% target. The deviation is attributed to distribution variance, demonstrating that the threshold generalizes well and that the RTL correctly implements sparsity-aware skipping. Figure 12 and Figure 18 show the corresponding waveform and terminal output.

IV. PLUS ALPHA IMPLEMENTATIONS

To facilitate rapid development and reliable testing of the dual-core design, a robust Makefile-driven flow and shell-based verification infrastructure were established.

A. Design Methodology and Verification Infrastructure

- **Production System Setup (Makefile-based Flow):** A centralized Makefile manages RTL simulation, GLS, and synthesis, with key parameters (e.g., target, clock, configuration) specified via command-line arguments for reproducible and rapid iteration. A summary is shown in Table IV. The `USER_DEFINES` variable (persisted in `USER_DEFINE_TASK_VARS`) enables flexible architectural exploration (e.g., `SFP_LONGDIV`, `SFP_MCP`, `SFP_THRESHOLD`) without modifying RTL or scripts. Since it propagates to both simulation and synthesis, it ensures consistency between RTL and final netlist.
- **Randomized Input Generation:** We support automatic generation of randomized test patterns for functional verification. The system detects missing pattern files and generates them on demand, allowing large-scale testing without manual data preparation. This improves coverage and helps uncover corner-case bugs under diverse input conditions.
- **.mingu Shell (Matrix INstruction Generation Utility):** A scriptable simulation shell built on top of the Makefile flow, providing a high-level interface (`writeQ`, `simulate`, etc.) to control the testbench. It supports configuration/execution phases, variable substitution, and `for`-loops for automated testing across Alpha and Step6.

For example, Fig. 22 shows generation of 100 randomized patterns and output verification using `.mingu`. Its well-defined interface allows high-level commands to map directly to hardware states, enabling scalable testing from single-core RTL to full-chip Post-PnR without new testbenches.

B. PnR Statistics

- 0 DRC violations were obtained for Steps 1,3 and 6
- PnR for Step4 and Step5 was closed with less than 5 DRC violations
- Near 0 WNS was achieved for both setup and hold time for Step5 and where we implemented our optimisations

V. DELIVERABLE PATH

Table II contains the paths to our submission.

TABLE II: Deliverable Path Summary

Step	Path
1	/home/linux/ieng6/ECE260B_WI26_A00/c8chiu/ECE260B-Final-Project/workspace/Step1
2	/home/linux/ieng6/ECE260B_WI26_A00/c8chiu/ECE260B-Final-Project/workspace/Step2
3	/home/linux/ieng6/ECE260B_WI26_A00/c8chiu/ECE260B-Final-Project/workspace/Step3_norm
4	/home/linux/ieng6/ECE260B_WI26_A00/c8chiu/ECE260B-Final-Project/workspace/Step4_hier
5	/home/linux/ieng6/ECE260B_WI26_A00/c8chiu/ECE260B-Final-Project/workspace/Alpha
6	/home/linux/ieng6/ECE260B_WI26_A00/c8chiu/ECE260B-Final-Project/workspace/Step6
Final Version	

VI. CONCLUSION

This project successfully demonstrates the design, implementation, and physical realization of a dual-core attention accelerator. By partitioning the system into asynchronous domains, we enabled flexible inter-core data exchange. Through the introduction of a Sparsity-Aware SFP engine, computations on insignificant data are bypassed, promoting energy efficiency. Most importantly, critical timing bottlenecks were identified and resolved via rigorous RTL repipelining, ensuring the design easily meets the 1.0ns clock constraint in a TSMC 65nm process. The progressive, step-by-step verification methodology guaranteed robustness from the behavioral level down to post-PnR gate-level simulations, confirming the design's readiness for physical fabrication.

TABLE III: Performance Comparison

	Power(VCD)	Setup WNS	Hold WNS	DRC
STEP 1	51.59 mW	-1.005		0
STEP 3	61.22 mW	-22.472		0
STEP 4	29.68 mW	-9.106	-4.713	1
STEP 5	54.21 mW	0.001	0.037	4
STEP 6	52.59 mW	0.004		0

TABLE IV: Supported Makefile Commands and Parameters

Command / Parameter	Description
make sim	Runs RTL simulation using iverilog and vvp. Supports SV via -g2012.
make gls	Runs gate-level simulation using post-synthesis netlists (syn/gate/*.out.v) and PDK.
make syn	Runs Design Compiler synthesis using run_dc.tcl.
make sim-shell	Launches the interactive 'mingu' shell for testbench execution.
make parse	Parses synthesis reports and outputs a summary of area, power, and timing.
TARGET=<name>	Specifies the design target (e.g., fullchip, core, mac, sfp_row).
SYN_EFFORT=<level>	Sets synthesis optimization effort: low, medium, or high (default).
CYCLE=<val>	Overrides the default clock period during simulation.
USER_DEFINES=<...>	Passes custom Verilog macros (e.g., SFP_LONGDIV) to both simulation and synthesis.

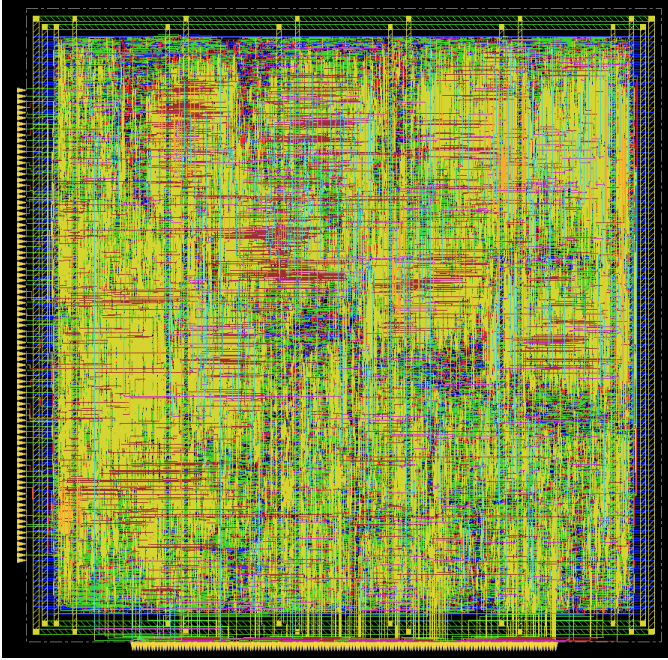


Fig. 1: Step 1 Single-Core Layout (post-PnR).

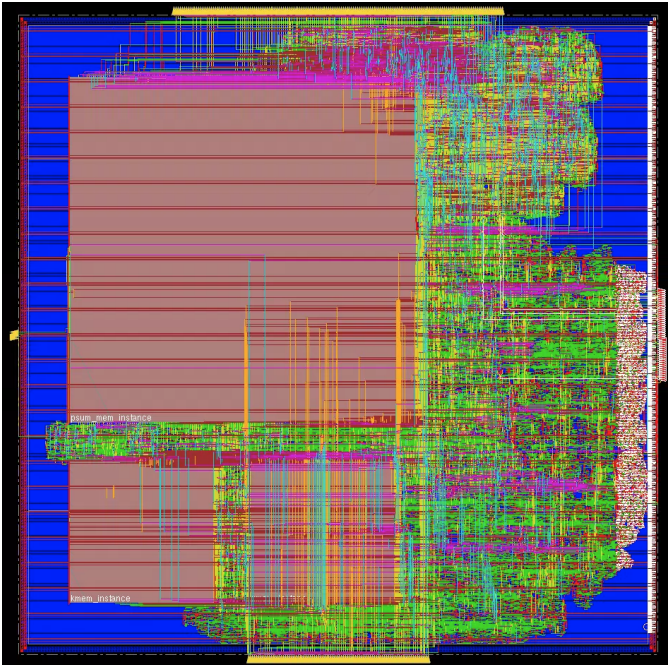


Fig. 2: Hierarchical Core Layout.

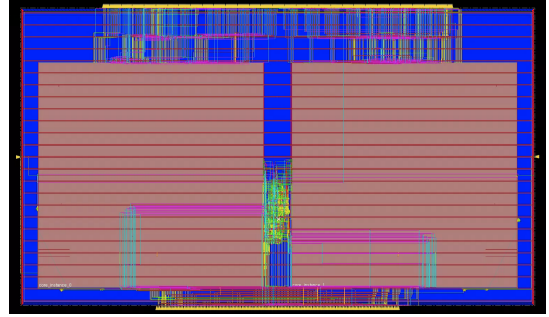


Fig. 3: Dual-Core Fullchip Layout (Step 4).

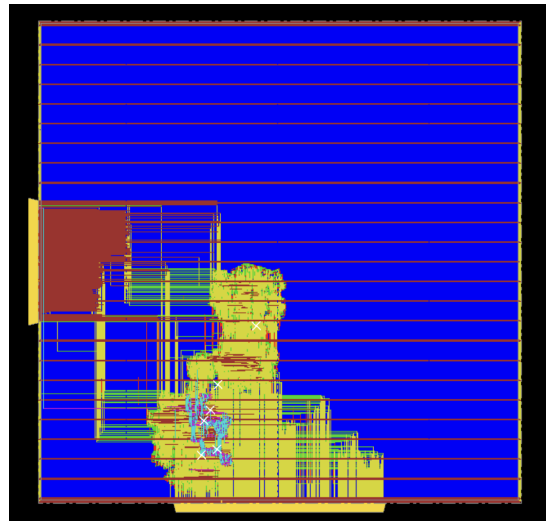


Fig. 4: Step 5 Fullchip Layout.

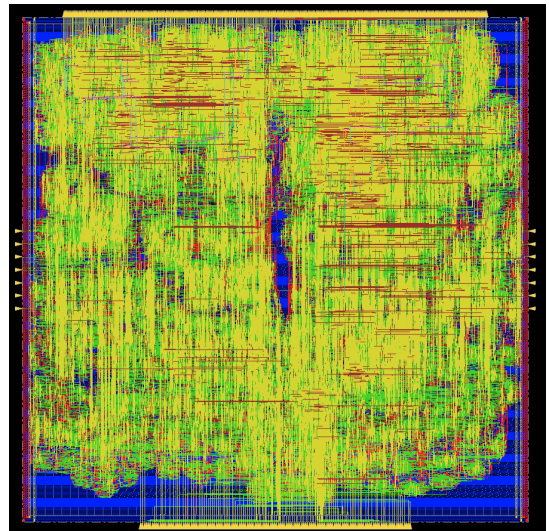


Fig. 5: Sparsity-Aware Attention Dual-Core Layout.

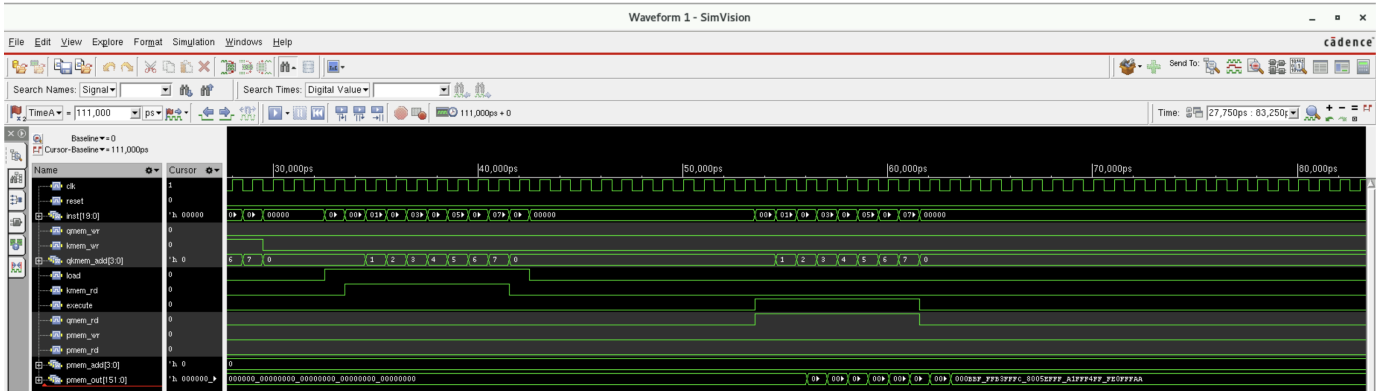


Fig. 6: Step 1 Single-Core GLS Waveform.

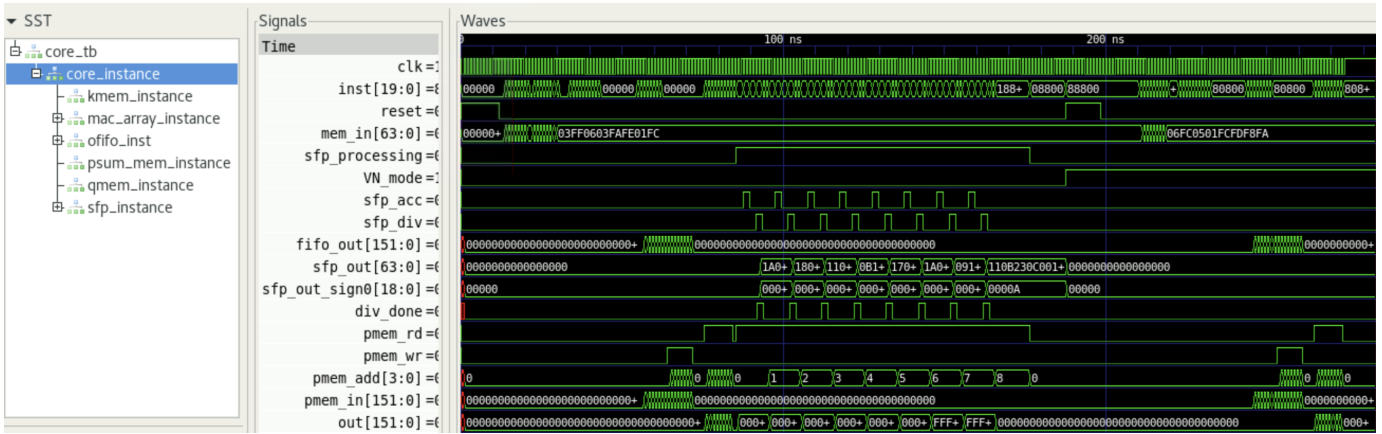


Fig. 7: Step 2 Simulation Waveform demonstrating functional correctness of Output Normalization.

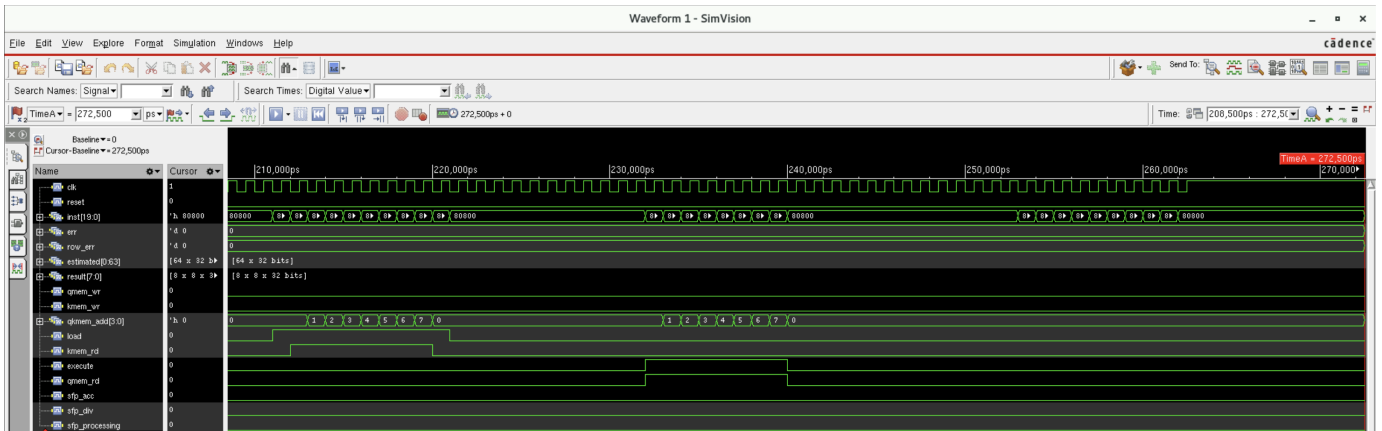


Fig. 8: Step 3 Hierarchical Core GLS Waveform.

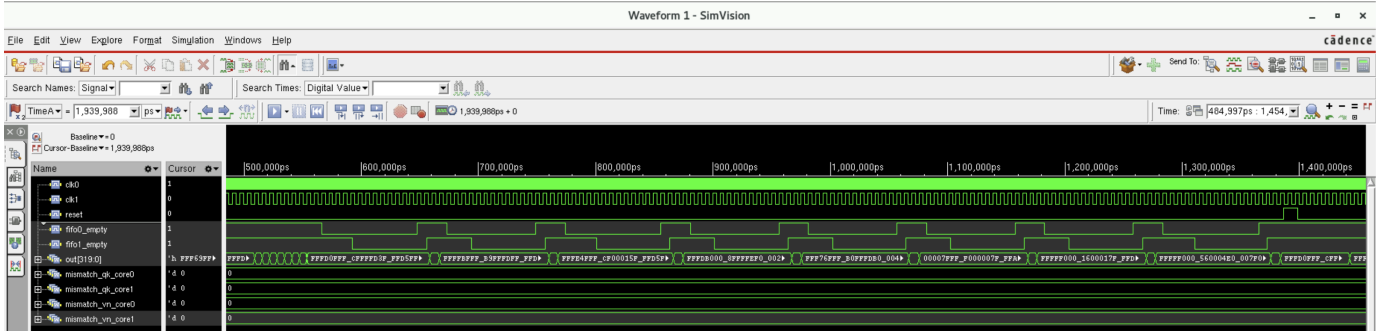


Fig. 9: Dual-Core Non-Lockstep GLS Waveform.

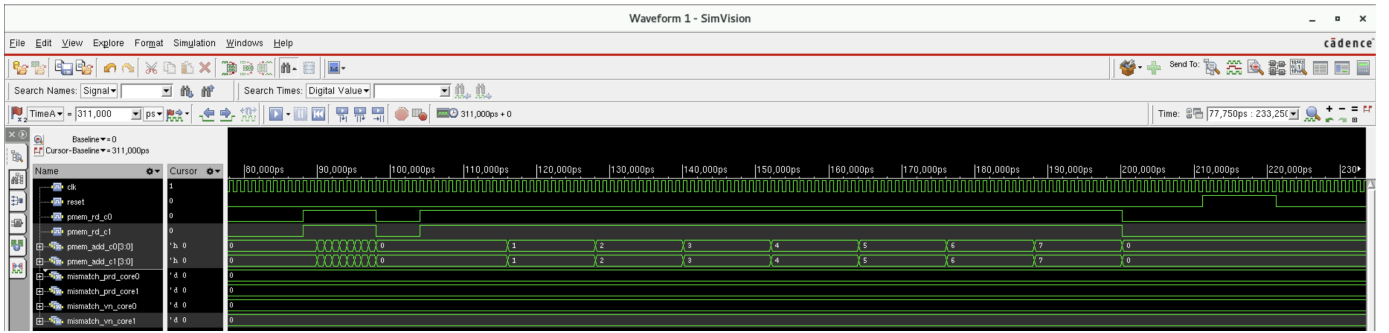


Fig. 10: Dual-Core Lockstep GLS Waveform.

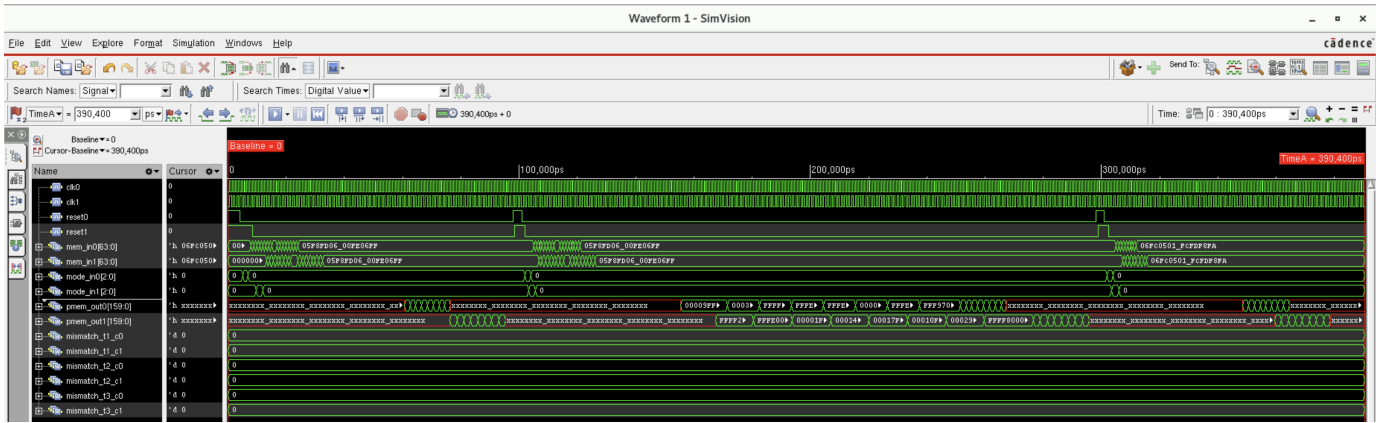


Fig. 11: Step 5 GLS Waveform.

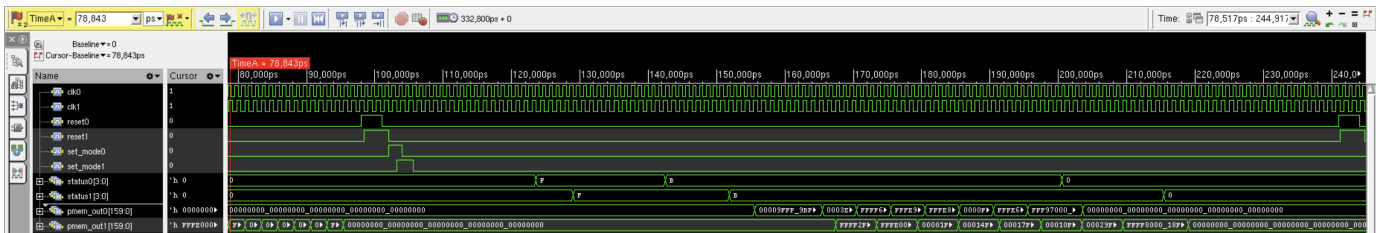


Fig. 12: Sparsity-Aware Attention Dual-Core GLS Waveform.


```

xcelium> run
#### 0 data txt reading ####
#### R data txt reading ####
#### Queue writing ####
#### Kmem writing ####
#### R loading to MLC ####
#### Execute ####
OK phase verification
[0] RTL: col0 col1 col2 col3 col4 col5 col6 col7
golden: 93 -20 -7 -7 94 -48 -45 -63 -86
[1] RTL: 52 -21 -73 94 24 -5 -33 -41 -23
golden: 52 -21 -73 94 24 -5 -33 -41 -23
[2] RTL: 38 38 27 50 50 63 -28 -28 21 -3 -3 -55
golden: 38 38 27 50 50 63 -28 -28 21 -3 -3 -55
[3] RTL: 33 -74 17 125 125 -37 -17 -27 -27 -51
golden: 33 -74 17 125 125 -37 -17 -27 -27 -51
[4] RTL: 80 80 -10 -10 13 13 -138 -37 -90 -90 -56
golden: 80 80 -10 -10 13 13 -138 -37 -90 -90 -56
[5] RTL: 42 1 26 68 7 7 7 -8 -45
golden: 42 1 26 68 7 7 7 -8 -45
[6] RTL: -19 -19 48 99 99 -7 -1 23 23 46 -20
golden: -19 -19 48 99 99 -7 -1 23 23 46 -20
[7] RTL: -63 41 128 -46 -1 78 74 37 37
golden: -63 41 128 -46 -1 78 74 37 37
PASS: all 8 x 8 results correct
Simulation complete via $finish(1) at time 111 NS + 0
/netlist/step1_tb.v:238 #10 $finish;

```

Fig. 13: Step 1 Single-Core GLS Terminal Output.

```

#### execute ####
VN phase verification start (checking pmem content)
#### sample pmem content & compare to golden ####
[0] RTL: col0 col1 col2 col3 col4 col5 col6 col7
golden: 160 -103 155 232 -311 283 -242 -79
[1] RTL: 238 85 98 127 334 179 54 75
golden: 238 85 98 127 334 179 54 75
[2] RTL: -214 -333 -334 46 -430 -300 -281 -253
golden: -214 -333 -334 46 -430 -300 -281 -253
[3] RTL: -517 -433 -354 -314 -526 -534 -309 -381
golden: -517 -433 -354 -314 -526 -534 -309 -381
[4] RTL: -311 40 -233 -253 -196 -396 298 68
golden: -311 40 -233 -253 -196 -396 298 68
[5] RTL: -419 -414 -467 -366 -456 -465 -371 -432
golden: -419 -414 -467 -366 -456 -465 -371 -432
[6] RTL: -154 -21 -47 -73 140 -867 187 3
golden: -154 -21 -47 -73 140 -867 187 3
[7] RTL: -201 24 -35 -201 -351 42 -58 -38
golden: -201 24 -35 -201 -351 42 -58 -38
PASS: 8 rows x 8 cols all match estimated result
Simulation complete via $finish(1) at time 272500 PS + 0
/netlist/core_tb.v:603 #10 $finish;

```

Fig. 14: Step 3 Hierarchical Core GLS Terminal Output.

```

##### FINAL SUMMARY #####
OK -- CORE0: 0 CORE1: 0 mismatch(es)
VN -- CORE0: 0 CORE1: 0 mismatch(es)
PASS: 8 rows x 8 cols all correct
Simulation complete via $finish(1) at time 1939988 PS + 0
/netlist/fullchip_sepclk_tb.v:258 #10 $finish;

```

Fig. 15: Dual-Core Non-Lockstep GLS Terminal Output.

```

##### OVERALL SUMMARY #####
Mode: LOCKSTEP (clk0 = clk1 = 1.0 GHz, 50% duty cycle)
OK phase -- CORE0: 0 CORE1: 0 mismatch(es)
VN phase -- CORE0: 0 CORE1: 0 mismatch(es)
>>> ALL TESTS PASSED <<<
Simulation complete via $finish(1) at time 311 NS + 0
/netlist/fullchip_lockstep_tb.v:615 #10 $finish;
xcelium>

```

Fig. 16: Dual-Core Lockstep GLS Terminal Output.

```

#### All tests complete, checking results ####
Test 1 (OK) -- C0: 0 C1: 0 mismatch(es)
Test 2 (Norm) -- C0: 0 C1: 0 mismatch(es)
Test 3 (VN) -- C0: 0 C1: 0 mismatch(es)
PASS: 8 rows x 8 cols all tests match estimated result
Simulation complete via $finish(1) at time 390400 PS + 0
/netlist/fullchip_tb.v:237 #10 $finish;
xcelium>

```

Fig. 17: Step 5 GLS Terminal Output.

```

#### All tests complete, checking results ####
Test 1 (OK) -- C0: 0 C1: 0 mismatch(es)
Test 2 (Norm) -- C0: 0 C1: 0 mismatch(es)
Test 3 (VN) -- C0: 0 C1: 0 mismatch(es)
PASS: 8 rows x 8 cols all tests match estimated result
Simulation complete via $finish(1) at time 332800 PS + 0
/netlist/fullchip_tb.v:255 #10 $finish;
xcelium>

```

Fig. 18: Sparsity-Aware Attention Dual-Core GLS Terminal Output.

Setup	Corner = TC Target Freq = 1G Fullchip.v	Pipelined Long Div	MCP / Operator
RTL Design	Latency	6cyc	10cyc
Timing	BC Hold WNS	0.068	0.060
	WC Setup WNS	0.000	0.000
	TC Setup WNS	0.000	0.000
Area(um^2)	Total	291242.5	376123.7
	Combinational	133760.2	228015.0
	Noncombinational	157482.4	148108.7
	Buf/Inv	4601.2	15221.2
Power(mW)	Total	65.2	67.5
	Internal	57.6	59.6
	Switching	6.1	6.1
	Leakage	1.5	1.8
Observation	Syn Time(min)	10	45
Validation	GLS	Pass	Pass
Decision		Go for PnR!	

Fig. 19: Synthesis result of core with different division module.

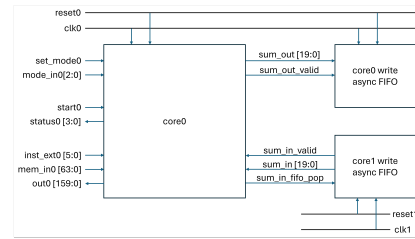


Fig. 20: Core interface redesign.

Signal Name	Dir	Width	Category	Description	Encoding / Behavior
set_mode	In	1	CTRL	Mode latch trigger. 1-cycle pulse. Latch mode_in on rising edge	100: MULTI+PMEM 001: MULTI+NONM+PMEM
mode_in[2:0]	In	3	CTRL	Core operation mode & result destination	010: MULTI+NONM+PMEM 011: MULTI+NONM+PMEM+PMEM others: Reserved, will not take effect
start	In	1	CTRL	Start execution 1-cycle pulse. Ignored if Core busy=1	
status[3:0]	Out	4	STATUS	Core busy 1: encoding, 0: idle	
status[2]	Out	1	STATUS	QMEM locked 1: QMEM in use, ignore Ext mem command QMEM_WRITE	
status[1]	Out	1	STATUS	QMEM locked 1: QMEM in use, ignore Ext mem command QMEM_WRITE	
status[0]	Out	1	STATUS	PMEM locked 1: PMEM in use, ignore Ext mem command PMEM_WRITE	
inst_ext[5:0]	In	6	MEM	Ext mem command	00: NO_OP 01: QMEM_WRITE 10: QMEM_WRITE 11: PMEM_READ
inst_ext[5:2]	In	4	MEM	Memory address. Address for external access	
mem_in[63:0]	In	64	MEM	Write data input. Used for QMEM/QMEM_WRITE	
out[159:0]	Out	160	MEM	Read data output. Output of PMEM read	

[Notes]
MULT: Result = QMEM*Transpose(QMEM)
MULT+NONM: Result = Norm(QMEM*Transpose(QMEM))

Fig. 21: Available core instruction/operation specification.

APPENDIX A

MINGU SHELL COMMAND REFERENCE

The .mingu shell abstracts the complexity of cycle-accurate testbenches into a high-level scripting language. Below is the complete reference of supported commands, categorized by execution phase.

A. Phase 1: Shell Configuration Commands

These commands must be executed **before** `fix_set`. They configure the underlying simulation backend, Makefile parameters, and output preferences.

- `set_scope <core|fullchip>`: Selects the hardware hierarchy to simulate. Determines whether to instantiate a single core or the dual-core fullchip.
- `set_sim_stage <sim|gls|post_sim>`: Sets the simulation backend.
 - `sim`: RTL behavioral simulation.
 - `gls`: Gate-level simulation using synthesized netlist.
 - `post_sim`: Post-PnR gate-level simulation with delay annotations via `xrun`.
- `set_clock_period <ns0> [ns1]`: Configures the half-cycle clock periods. If `ns1` is omitted, it defaults to `ns0` (lockstep). Example: `set_clock_period 1 1.2`.
- `set_dump_vcd <0|1>`: Toggles waveform dumping (1 for ON, 0 for OFF). Disabling VCD significantly speeds up massive test runs.
- `set VAR value`: Defines a shell variable that can be substituted later using `${VAR}`. Example: `set PATTERN sw/pattern/random100`.
- `fix_set`: **Required.** Locks all configurations, triggers compilation of the hardware design, and transitions the shell into the execution phase.

B. Phase 2: Hardware Execution Commands

These commands are executed **after** `fix_set`. They map directly to the unified controller interface of the chip.

- `set_exec_target <MODE>`: Sets the execution mode of the hardware controller (mapped to `mode_in`).
 - `CORE_MODE_MULT_save_to_PMEM`: Standard $Q \times K^T$.
 - `CORE_MODE_MULT_NORM_save_to_PMEM_and_KMEM`: Computes $Q \times K^T$, applies Softmax, and saves to both Partial-Sum memory and Key memory for subsequent computation.
- **Memory Writers**: Loads text-based integer matrices into the internal SRAMs via the `inst_ext` and `mem_in` ports.
 - `writeQ <filepath>`: Loads data into QMEM.
 - `writeK0 <filepath>`: Loads data into Core 0's KMEM.
 - `writeK1 <filepath>`: Loads data into Core 1's KMEM.
- `simulate`: Asserts the start signal and waits until the core's `status[3]` (busy flag) goes low. Internally, the testbench computes the golden model equivalent for validation.
- **Golden Checkers**: Reads out the data from the PMEM via the `inst_ext` port and compares it against the golden model.
 - `verifypmem`: Checks PMEM results for all active cores and prints PASS/FAIL statistics.
 - `verifypmem0` / `verifypmem1`: Specific checks for Core 0 or Core 1.
- `reset`: Asserts the hardware reset signal for two cycles, clearing internal states.
- `for <var> = <start> to <end> ... endfor`: Iterates block of execution commands. Variables inside the loop can be referenced using `${var}`. Nested loops are not supported.
- `exit`: Terminates the simulation gracefully and prints a summary of mismatches.

TABLES AND FIGURES

TABLE V: Summary of standard .mingu scripts and their purposes

Script Name	Purpose & Features
in-n-out-fullchip.mingu	Basic RTL simulation pipeline executing $Q \cdot K^T$, Norm, and $V \cdot \text{Norm}^T$ sequentially on a single static test pattern.
in-n-out100-fullchip.mingu	Large-scale automated RTL simulation utilizing the <code>for</code> loop. It automatically triggers the python/shell pattern generator to create 100 sets of randomized Q, K, and V matrices and verifies the hardware against a software golden model continuously.
in-n-out500-fullchip.mingu	Stress-testing variant scaling the loop and automatic pattern generation to 500 unique datasets.
in-n-out*-fullchip-gls.mingu	Post-PnR validation scripts. By simply changing <code>set_sim_stage</code> to <code>post_sim</code> , the exact same test sequence is piped into <code>xrun</code> with the synthesized and routed netlist (with back-annotated delays or threshold logic).

```

# =====
# in-n-out100-fullchip.mingu - Batch run random100 pattern sets (0..99) [fullchip_shell]
#
# Dual-core: writeK0/writeK1. Uses same kdata for both cores (or gen_random_patterns --dual_core).
#
# USAGE: ./sim_shell.sh -f in-n-out100-fullchip.mingu
# (auto-runs make gen-patterns if sw/pattern/random100/ is missing)
# =====

set PATTERN sw/pattern/random100
set_output_dir sim/waveform
set_sim_stage sim
set_clock_period 1
set_sim_define SFP_LONGDIV
set_sim_target fullchip_shell
set_dump_vcd 0
fix_set

for i = 0 to 99
  reset

  # Test 1: Q*K^T
  set_exec_target CORE_MODE_MULT_save_to_PMEM
  writeQ $(PATTERN)/qdata_$(i).txt
  writeK0 $(PATTERN)/kdata_$(i).txt
  writeK1 $(PATTERN)/kdata_$(i).txt
  simulate
  verifypmem

  # Test 2: Norm(Q*K^T)
  set_exec_target CORE_MODE_MULT_NORM_save_to_PMEM_and_KMEM
  writeQ $(PATTERN)/qdata_$(i).txt
  writeK0 $(PATTERN)/kdata_$(i).txt
  writeK1 $(PATTERN)/kdata_$(i).txt
  simulate
  verifypmem

  # Test 3: V*Norm^T
  set_exec_target CORE_MODE_MULT_save_to_PMEM
  writeQ $(PATTERN)/vdata_$(i).txt
  simulate
  verifypmem

  inQuitive, 3 days ago + alpha integration finish, sim clean, gls clean
endfor

```

Fig. 22: in-n-out100-fullchip.mingu execution script demonstrating `for`-loops and automated testing.